

IGL233 : METHODE ORIENTEE OBJET UML

NIVEAU : BTS 2

SPECIALITE : GENIE-LOGICIEL

Année académique : 2024-2025

PAR M. TCHUIMEGNI YOUNBI JOEL
Enseignant-Chercheur

CONTENU :

- ✓ CHAPITRE 1 : RAPPELS SUR LES SYSTÈME D'INFORMATION
- ✓ CHAPITRE 2 : RAPPELS SUR LES NOTIONS DE GENIE LOGICIEL
- ✓ CHAPITRE 3 : INTRODUCTION A LA MODELISATION OBJET UML

CHAPITRE 1 : RAPPELS SUR LES SYSTÈME D'INFORMATION

Partie 1- Les concepts de base sur les SI

Un **Système d'Information** (SI) est un ensemble de ressources matérielles, logicielles et humaines permettant d'acquérir, stocker, traiter et restituer les informations au sein d'une organisation. Il sert à traiter l'information et à la véhiculer entre le système de pilotage et le système opérant. Un **Système d'Information est dit Automatisé** (SIA) lorsque les traitements sont effectués au moyen des outils NTIC (ordinateurs, tablettes,...). Comme exemples de SIA nous avons : les brasseries du Cameroun, les Banques, la société CAMLAIT...

1- Description des systèmes d'une entreprise

Une **entreprise** est une Unité économique juridiquement autonome et ayant pour fonction principale la production des biens et des services pour le marché.

L'entreprise est organisée autour de trois grands systèmes qui sont : le système opérant, de pilotage et d'information

a) Le système de pilotage

Appelé également système de décision, c'est la tête pensante de l'entreprise. Il prend les grandes décisions stratégiques, fixe les objectifs et les moyens de les atteindre. Le système de pilotage influence sur tous les niveaux de l'entreprise.

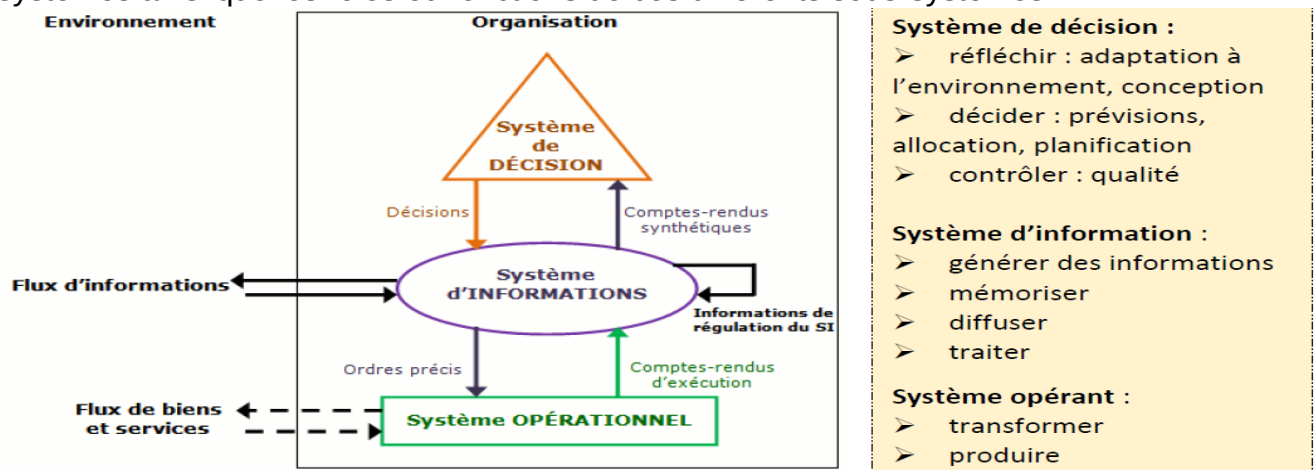
b) Le système opérant ou opérationnel

Il est constitué du personnel chargé d'exécuter les tâches données par le système de pilotage. C'est la partie de l'entreprise qui assure la production des biens et services.

c) le système d'information

Il sert à traiter l'information et à la véhiculer entre le système de pilotage et le système opérant.

La figure ci-après nous montre les flux ou échanges entre les différents sous-systèmes ainsi que les rôles ou fonctions de ces différents sous-systèmes. .



2- Quelques composants d'un SI

Le SI est organisé autour de plusieurs composants qui sont : les ressources humaines, matérielles, logicielles, les procédures et données.

a) Les ressources humaines

Il s'agit de l'ensemble des personnels (personnes) qui se trouvent au sein de l'entreprise. Cette ressource est répartie selon les rôles de chacun autour du SI.

b) Les ressources matérielles

Il s'agit de l'ensemble de biens matériels de production qui se trouvent au sein de l'entreprise et qui servent au traitement de l'information. Par exemples :

- les ordinateurs, les serveurs, les imprimantes et photocopieurs
- les supports de stockage (Disque dur, clé USB, etc.),
- les équipements d'interconnexion (commutateur, routeur, concentrateur, modem,),
- les dispositifs de sécurité (caméra de surveillance, lecteur de badge, lecteur de carte, lecteur d'emprunte, etc.),

c) Les ressources logicielles

Elles comprennent : Les logiciels de base ou « système » : les programmes qui gèrent et commandent le matériel informatique tels que les systèmes d'exploitation. Les logiciels d'application : les programmes destinés à un traitement particulier requis par l'utilisateur tels que le programme de la gestion de la paie, de la facturation et la comptabilité.

3- Les fonctions d'un SI

Il existe donc 4 fonctions principales d'un SI :

- ✓ **Collecter** : c'est à partir de là que naît la donnée, qu'on acquière les informations provenant de l'environnement interne ou externe à l'entreprise.
- ✓ **Stocker / mémoriser** : dès que l'information est acquise, le système d'information la conserve pour pouvoir être disponible et conservée dans le temps. Cela implique la mise en œuvre des moyens techniques et organisationnels (méthodes d'archivage par exemple) pour stocker les informations de manière durable et stable (sous forme de bases de données principalement).
- ✓ **Transformer/traiter** : cette phase permet de transformer l'information et choisir le support adapté pour traiter l'information. Ici on construit de nouvelles informations en modifiant le fond ou la forme.
- ✓ **Diffuser** : Il s'agit de la mise à disposition de l'information pour ceux qui en ont besoin (environnement interne ou externe) au moment où c'est nécessaire, sous une forme directement exploitable.

4- Quelques Méthodes et langage De Conception Des SI

L'informatisation d'une organisation passe par la description des procédures et des données manipulées par celle-ci, grâce à un formalisme bien élaboré. Ce formalisme prend corps à travers les méthodes de conception présentées dans le tableau suivant :

Sigle	Signification	Brève description
MERISE	Méthode d'Étude et de réalisation Informatique pour les Systèmes d'Entreprise	Adapter pour les projets d'entreprise se limitant à un domaine précis (méthode française)
UML	Unified Modeling Language	Permet de modéliser des applications construites à l'aide d'objets.
OMT	Object Modeling Technic	Utilise le même formalisme pour le processus d'analyse et de conception
SADT	Structured Analysis Design Technic	Décrit les tâches d'un projet, leurs interactions ainsi que le système concerné

Toutes ces méthodes permettent de concevoir des systèmes d'information normalisés.

Partie 2- La Conception D'un SI

I- LE MODÈLE CONCEPTUEL DES DONNÉES (MCD)

La conception d'un système d'information passe par la mise sur pied de modèles à travers une activité de modélisation.

Un modèle est une représentation simplifiée d'un système, généralement sous forme de diagramme, en vue de faciliter sa compréhension ou sa gestion par les outils informatiques.

La modélisation est le processus de construction d'un modèle après une étude tenant compte des objectifs assignés.

Le Modèle Conceptuel des Données (MCD) est une représentation graphique servant à décrire l'architecture des données du système d'information d'une

organisation. Il s'appuie sur des concepts tels que : l'entité, les propriétés de l'entité, les relations et les cardinalités.

1. LES ENTITÉS ET LEURS PROPRIÉTÉS

L'entité est l'un des concepts fondamentaux du MCD. Chaque entité est pourvue de propriétés.

a) Définitions et description d'une entité et ses propriétés

Une Entité est tout objet concret (matériel) ou abstrait (immatériel) ayant une existence propre et conforme aux besoins de gestion d'une organisation.

Exemple d'entité : dans le système constitué par un garage où des mécaniciens réparent les véhicules déposés par des clients, l'analyse de ce système nous permet de citer les entités suivantes : MECANICIEN, VEHICULE, CLIENT.

Une entité est caractérisée par ses propriétés.

Une propriété ou attribut est une information élémentaire caractérisant une entité et présentant un intérêt pour le système étudié.

Exemple de propriétés d'un véhicule : le numéro d'immatriculation, la marque, le modèle, la couleur, etc.

L'entité n'existe que si on peut lui trouver plusieurs exemplaires. Un exemplaire de l'entité est appelé une occurrence ou une instance. Il s'agit d'un cas réel d'existence de l'entité.

Exemple d'occurrence : M. Jean est une occurrence de l'entité CLIENT. Le véhicule de M. Jean est une occurrence de l'entité VEHICULE. Ses propriétés sont : numéro d'immatriculation : OU 5418 AD, marque : Toyota, modèle : Avensis berline, couleur : verte olive.

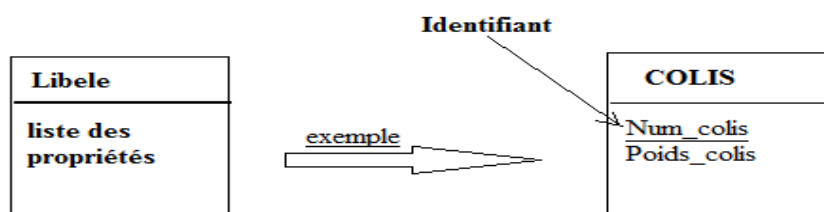
Chaque occurrence d'une entité doit avoir une propriété dont la valeur unique la distingue des autres occurrences. La propriété correspondant à cette valeur unique est appelée un identifiant.

Un identifiant est une propriété particulière dont la valeur est unique pour chaque occurrence de l'entité. Cette propriété apparaît toujours soulignée pour marquer sa singularité.

Exemple d'identifiant : l'entité VEHICULE a pour identifiant la propriété numéro d'immatriculation dont la valeur est unique. C'est le cas du numéro OU 5418 AD qui concerne le véhicule de M. Jean.

b) Représentation d'une entité

Les entités sont représentées par un rectangle. Ce rectangle est séparé en deux parties: la partie haute contient le libellé et celle du bas contient la liste des propriétés de l'entité. L'identifiant apparaît souligné dans cette modélisation.



2. LES RELATIONS ET LES CARDINALITÉS

La relation ou association est l'autre concept fondamental du MCD. Chaque relation donne lieu à l'établissement des cardinalités.

a) La relation (association) entre des entités

Une relation ou association est un lien sémantique entre deux ou plusieurs entités. Selon le nombre d'entités liées :

- ✓ La relation récursive ou réflexive relie une entité à elle-même ;
- ✓ La relation binaire relie deux entités ;
- ✓ La relation ternaire relie trois entités ;
- ✓ La relation n-aire relie n entités.

Exemple de relation dans le système constitué par un garage où des mécaniciens réparent les véhicules déposés par des clients, nous pouvons identifier deux relations : la relation REPARER entre MECANICIEN et VEHICULE (des mécaniciens réparent les véhicules) et la relation DEPOSER entre CLIENT et VEHICULE (les véhicules sont déposés par des clients).

b) La cardinalité d'une relation

La cardinalité représente le nombre minimal et maximal d'occurrences de l'entité qui peuvent participer à une occurrence de l'association. Les cardinalités doivent obéir aux règles de gestion de l'entreprise (règles imposées par le gestionnaire).

✓ La borne minimale (généralement 0 ou 1) décrit le nombre minimum de fois qu'une occurrence de l'entité peut participer à une relation.

✓ La borne maximale (généralement 1 ou n) décrit le nombre maximum de fois qu'une occurrence de l'entité peut participer à une relation.

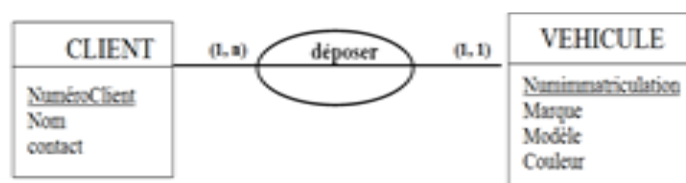
Exemple de cardinalité : dans l'exemple du garage, pour être considéré comme client, il faut déposer au minimum, un (1) véhicule et au maximum plusieurs (n) véhicules. De façon réciproque, un véhicule est déposé par un (1) et un (1) seul client, ainsi donc :

✓ Un CLIENT dépose un ou plusieurs VEHICULE ; la cardinalité coté CLIENT se note donc (1, n) ;

✓ Un VEHICULE est déposé par un et un seul CLIENT ; la cardinalité coté VEHICULE se note (1,1) ;

c) Représentation d'une relation et des cardinalités

Les relations sont représentées par des ellipses dont l'intitulé (un verbe à l'infinitif) décrit la relation existante entre les entités. On peut éventuellement ajouter des propriétés aux relations si le système étudié l'exige. Les cardinalités sont mentionnées par un couple du côté de l'entité à considérer. La cardinalité minimale est représentée en premier, et la maximale en second. En considérant l'exemple ci-dessus : un client dépose un ou plusieurs véhicules (1, n) et un véhicule est déposée par un et un seul client (1,1), donne la représentation ci-contre :



d) Les types de relations selon les cardinalités

Selon les cardinalités, on distingue 3 types de relations entre deux entités : les relations de type (1:1), de type (1:n) et de type (n:n).

i. Les relations de type (1 à 1) ou (1:1) souvent appelée relation fils-fils. Ici, les cardinalités maximales sont à 1 de chaque côté. Ce cas rare traduit quelque fois une mauvaise conception.

Les exemples de telles relations sont celles dont les cardinalités sont ((1,1) à gauche et (1,1) à droite) ou idem ((0,1)–(0,1)) ou ((0,1)-(1,1)).

ii. Les relations de type (1 à n) ou (1:n) souvent appelée relation hiérarchique ou relation père-fils. Ici, l'une des cardinalités maximales est n, tandis que l'autre est à 1.

Les exemples de telles relations sont celles dont les cardinalités sont ((1, n) à gauche et (0, 1) à droite) ou ((1,n)–(1,1)) ou ((0, n)-(0, 1) ou ((0,n)–(1,1)).

iii. Les relations de type (n à n) ou (n:n) souvent appelée relation non hiérarchique ou père-père pour les cardinalités maximales à n de chaque côté, c'est-à-dire ((1, n)-(1, n) ou ((0, n) – (0, n)) ou ((0, n)-(1, n))

3. MÉTHODE D'ÉLABORATION D'UN MCD

Pour construire un MCD, il faut le faire étape après étape. Il s'agit d'abord de lire attentivement le texte soumis ou d'écouter les acteurs lors d'une interview, dans le but de :

- ✓ Rechercher les Entités ;
- ✓ Rechercher les propriétés de chaque entité en se limitant aux informations reçues du texte ;
- ✓ Repérer l'identifiant de chaque entité dans le texte sinon l'ajouter si cela n'est pas mentionné ;
- ✓ Rechercher les associations entre les entités (verbe infinitif);
- ✓ Affiner les associations en les lisant dans les deux sens afin de ressortir les cardinalités, conformément aux règles de gestion de l'entreprise ou de l'organisation (exemple CLIENT déposer VEHICULE : on se pose les questions - un client dépose combien de véhicules ? Un véhicule est déposée par combien de clients ?)
- ✓ Représenter correctement les entités, les propriétés ; les associations ; les cardinalités.

II- LE MODÈLE LOGIQUE DES DONNÉES (MLD)

La modélisation avec la méthode MERISE nous conduit à identifier et représenter dans les premières phases d'analyse les entités et leurs propriétés, mais aussi les relations qu'elles entretiennent. Le schéma résultant est le modèle conceptuel de données (MCD). Le MCD, sous sa forme graphique ne peut pas être représenté tel quel dans un logiciel de gestion des données (SGBDR). On doit le transformer en un Modèle Logique de Données (MLD) ou Modèle Relationnel de données beaucoup plus simple. Tous les SGBDR (Microsoft ACCESS, MySQL, ORACLE) implantent les bases de données à l'aide de ce type de modèle.

1. LES RÈGLES DE PASSAGE DU MCD AU MLD

Le passage du MCD au MLD se fait via 4 règles en fonction du (ou des) type(s) de lien(s) que comporte(nt) le MCD :

Règle 1 : Transformation des entités en tables, c'est-à-dire que chaque entité du modèle conceptuel devient une table dans le modèle logique et conserve les mêmes propriétés qui deviennent des champs ou colonnes. L'identifiant de l'entité devient la clé primaire de la table.

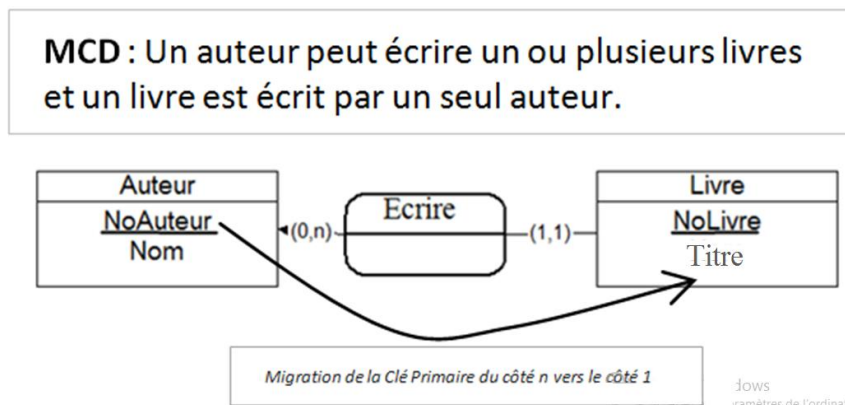
Exemple d'application : l'entité Client devient d'après la règle 1 devient une table et les attributs deviennent les attributs (champs de la table). L'identifiant de CLIENT devient la clé primaire (souligné).

CLIENT (NumClient, NomClient, NumTéléphone, AdresseClient);

CLIENT
<u>NumClient</u>
NomClient
NumTéléphone
AdresseClient

- **Règle 2 :** Destruction de la relation 1 : n, c'est-à-dire que dans les associations de type hiérarchique « un à plusieurs » l'association disparaît et l'identifiant de l'entité forte (coté n) migre dans l'entité faible (coté 1) pour devenir une clé étrangère. Les propriétés de l'association dans le cas où ils existent migrent également du même coté (de façon imagée, le père donne son nom à ses fils).

Exemple d'application : la relation hiérarchique (1:n) ou père-fils ci-dessous donne deux tables, Auteur et Livre et il y'a migration de l'identifiant NoAuteur de l'entité Père vers Livre (clé étrangère).



devient :

Auteur (NoAuteur, Nom) ;

○ NoAuteur est la clé Primaire de la table Auteur

Livre (NoLivre, Titre, #NoAuteur)

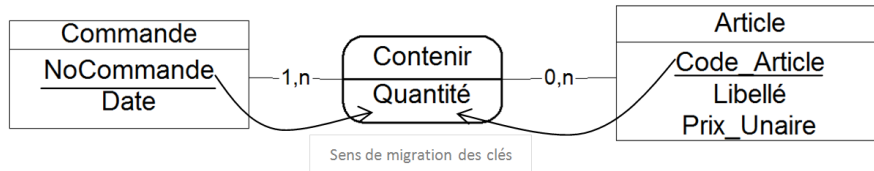
○ NoLivre est la clé primaire de la table Livre

○ NoAuteur est la clé étrangère de la table Livre

- **Règle 3 :** Transformation de la relation non hiérarchique n : n en table, c'est-à-dire que dans les associations de plusieurs à plusieurs, l'association devient une table, sa clé primaire sera la concaténation (combinaison) des identifiants de toutes les

entités qui interviennent dans l'association ; les propriétés de l'association deviennent également des attributs de ladite table.

- Exemple d'application : la relation de type plusieurs à plusieurs Contenir entre Commande et Article donne lieu à une table supplémentaire dont la clé primaire est un couple composé des deux clés primaires des deux tables.



Devient :

Commande (NoCommande, Date)

- NoCommande est la clé primaire de l'entité Commande

Article (Code Article, Libellé, Prix_Unaire)

- Code_Article est la clé primaire de l'entité Article

Contenir (NoCommande, Code Article, Quantité)

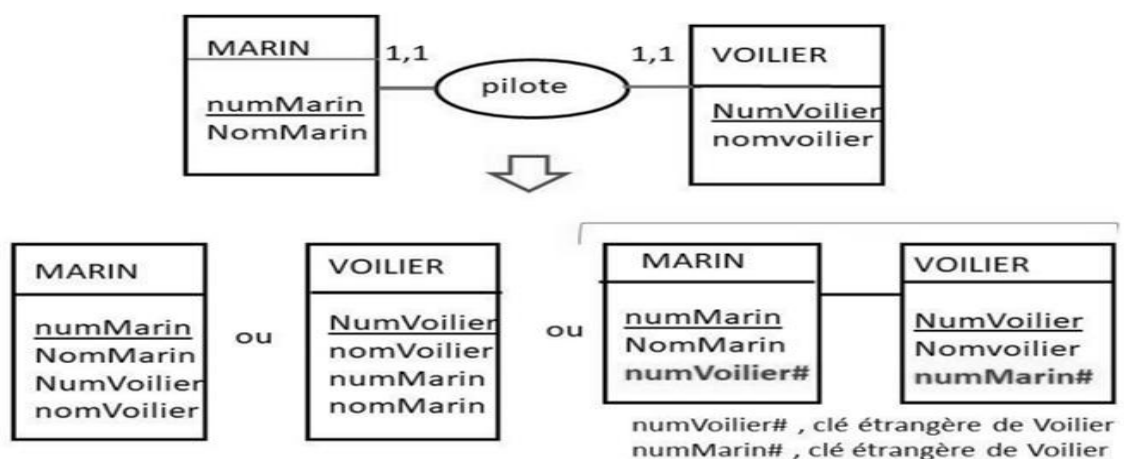
- le couple (NoCommande, Code_Article) est la clé primaire de l'entité contenir
- NoCommande est l'une des clés étrangères de l'entité Contenir
- Code_Article est l'autre clé étrangère de l'entité Contenir

- **Règle 4 : Destruction de la relation 1:1 et application de la contrainte d'unicité sur la clé étrangère**, c'est-à-dire que l'association disparaît, les identifiants migrent de part et d'autre des deux entités, l'entité la moins pertinente est également supprimée.

- Dans le cas d'une relation (1,1) vers (1,1) on peut :

- Fusionner les deux tables en choisissant le nom de l'entité la plus pertinente ;
- Garder les deux tables et faire glisser les clés primaires de part et d'autre pour avoir des clés secondaires.

Exemple d'application : à la course à la voile, un marin pilote un voilier et un voilier est piloté par un marin. Il y'a trois solutions :



- Dans le cas d'une relation (0,1) vers (1,1) on doit ajouter la clé étrangère du coté (1,1) ;
- Dans le cas d'une relation (0,1) vers (0,1) on peut :

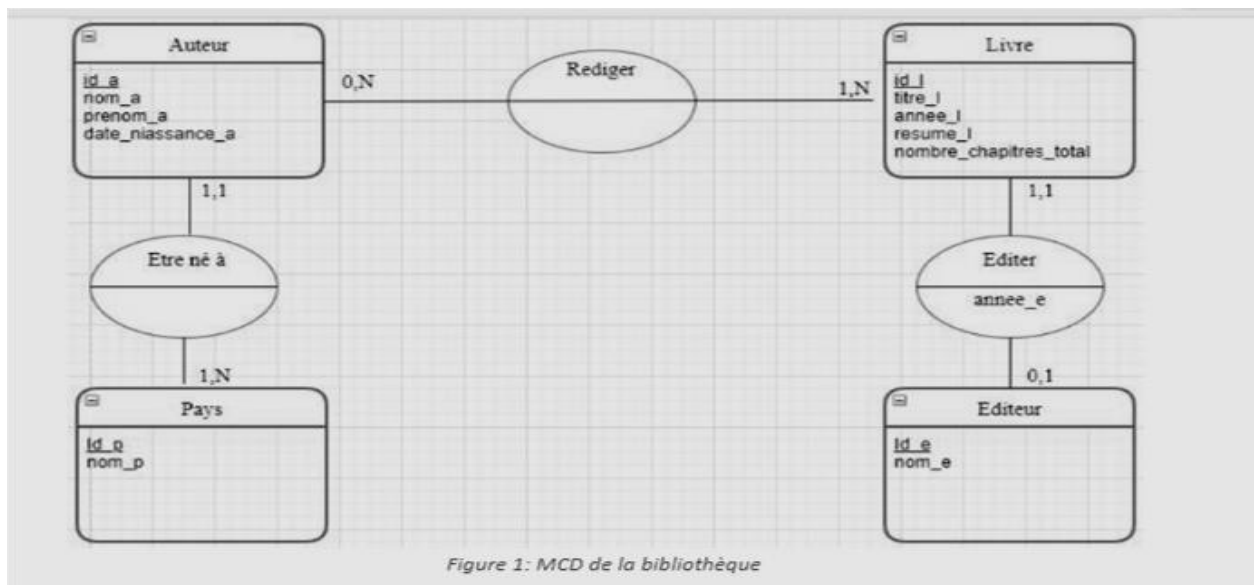
- Ajouter la clé secondaire sur l'une quelconque des tables, mais ajouter une contrainte d'unicité et de non vacuité (non nul) sur cette clé étrangère ;
- Créer une table supplémentaire comme dans le cas d'une relation n : n

NB. La normalisation des clés primaire et étrangère se fait avec :

- la clé primaire soulignée (exemple : NumClient) ;
- la clé étrangère précédée ou suivie du symbole dièse (#) (#codeCl).

2. QUELQUES EXEMPLES DE TRANSFORMATION DE MCD EN MLD

CAS PRATIQUE : Le proviseur du lycée bilingue souhaite mettre en place un SI afin d'améliorer la gestion de la bibliothèque. Un de vos camarades a dessiné dans son cahier le MCD de la figure suivante :



Transformation en MLD :

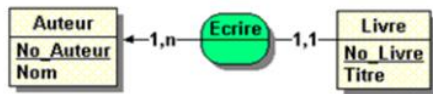
En appliquant les règles 1, 2, 3 ci-dessus, on obtient le schéma relationnel suivant :

- Pays** (Id_p, nom_p)
- Auteur** (Id_a, nom_a, prenom_a, date_naissance_a, #Id_P)
- Livre** (Id_l, Titre_l, Annee_l, Resume_l, nombre_chapitres_total)
- Rédiger** (Id_a, Id_l)
- Editeur** (Id_e, nom_e)
- Editer** (#Id_l, #Id_e, annee_e)

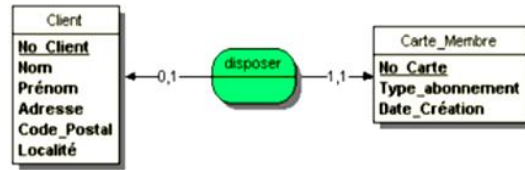
- En a) on a appliqué la règle 1 de la transformation des entités.
- En b) on a appliqué la règle 2 des associations de type 1 : n à l'association être né à en ajoutant la clé étrangère coté (1,1).
- En c) on a appliqué la règle 1
- En d) on a appliqué la règle 3 des associations n : n Rédiger
- En e) on a appliqué la règle 1
- En f) on a appliqué la règle 3, car l'association Editer a une propriété

Exercices d'application

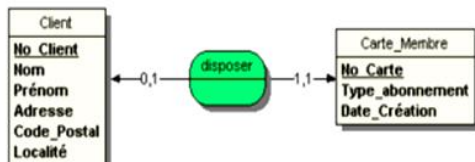
Exercice 1: traduire les quatre MCD suivants en MLD



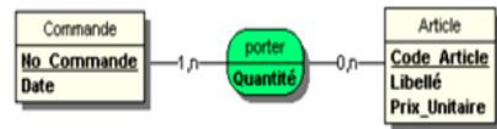
(a)



(c)



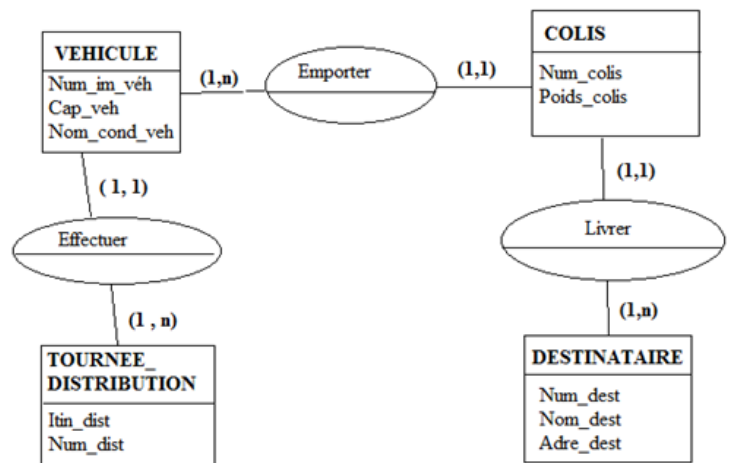
(b)



(d)

Exercice 2 : Considérons l'entreprise de distribution de colis dont le MCD suit :

1. Classifier les associations de ce MCD, en fonction des cardinalités.
2. Dédire le MLD correspondant.



Exercice 3 :

Une usine contient des machines qui peuvent fabriquer au moins un type de pièces. Chaque pièce peut être fabriquée par une ou plusieurs machines. Chaque type de machine est construit par un ou plusieurs fournisseurs. Le fournisseur peut construire une ou plusieurs marques de machines.

Proposer le MCD correspondant à cette description puis déduire le MLD.

Exercice 4 :

Dans une entreprise, on peut trouver un ou plusieurs départements, chaque département est identifié par un nom et caractérisé par une localisation. Un employé est caractérisé par un numéro, son nom, son grade, ce dernier peut travailler dans un ou plusieurs départements, dans chaque département existe au moins un employé. Donner le MCD, en précisant les attributs puis déduire le MLD

Exercice 5 :

On considère une bibliothèque contenant des ouvrages. Un ouvrage est caractérisé par un numéro, un titre, un auteur et un éditeur. La bibliothèque dispose d'un ou plusieurs exemplaires de chaque ouvrage. L'exemplaire est identifié par un numéro et caractérisé par

la date d'édition. Un exemplaire peut être emprunté par un ou plusieurs emprunteurs, ce dernier est identifié par un numéro et caractérisé par un nom, un téléphone. Un emprunteur peut emprunter un ou plusieurs exemplaires. Donner le MCD, en précisant les attributs puis déduire le MLD

Exercice 6 :

Créer le MCD permettant à un groupe de gérer les droits d'auteurs des livres publiés par ses différentes maisons d'éditions. Elle doit respecter les contraintes suivantes. Un livre peut être écrit par un ou plusieurs auteurs. Un auteur peut écrire un ou plusieurs livres. Un éditeur peut publier un ou plusieurs livres. Un livre est publié par un seul éditeur. Déduire le MLD.

Exercice 7 :

Pour la gestion de sa petite boutique. M. KAMGA fait appel à vous pour l'informatisation de son processus de vente des produits.

Le principe de fonctionnement des ventes est le suivant :

- Un client passe la commande concernant un certain nombre de produits ;
- Le nom, et le Téléphone du client sont recueillis et un identifiant numclient lui est attribué ;
- Sa commande est enregistrée avec un identifiant numCommande ainsi que la date ;
- Les produits stockés dans l'entreprise sont enregistrés avec un code qui les identifie, un libellé et un prix unitaire ;
- Une commande doit contenir des produits, et n'est passé que par un et un seul client ;
- Un client doit avoir passé au moins une commande pour être enregistré ;
- Certains produits peuvent n'avoir jamais été commandés;
- A l'achat d'un produit, la quantité commandée est enregistrée

A partir de la description ci-dessus et de vos connaissances en Systèmes d'informations de répondez aux questions suivantes :

1. Produire le MCD décrivant ce fonctionnement en représentant :
 - a. Les entités et leurs propriétés ;
 - b. Les relations entre entités ainsi que les éventuelles propriétés ;
 - c. Les cardinalités.
2. En appliquant les règles de passage du MCD vers le MLD, déduire le MLD correspondant au fonctionnement de ce système.

CHAPITRE 2 : RAPPELS SUR LES NOTIONS DE GENIE LOGICIEL

PARTIE 1 : ORIGINE, OBJECTIF ET DEFINITION DU GENIE LOGICIEL

1- ORIGINE DU GENIE LOGICIEL

La notion de «Génie Logiciel» a été proposé en 1968 lors de la conférence «Garmisch-Partenkirchen, Germany, 7th to 11th October 1968 » pour discuter de ce qui était alors appelé la «Crise du Logiciel».

- **A la fin des années 70**, on ne sait pas « faire » des logiciels (plusieurs tentatives de logiciels vont échouer)

- Les besoins et contraintes de l'utilisateur ne sont pas respectés (pas de facilité d'emploi, interfaces homme/machine inexistantes)
- Les coûts ne sont pas maîtrisés
- On relève régulièrement des retards dans la livraison des produits (non maîtrise du temps)
- On note une grande rigidité dans les solutions mises en place
- Quelques fois on arrive à des incohérences pendant l'utilisation
- On est aussi confronté au manque de performance et de robustesse (fiabilité)
- Les codes sont difficiles à maintenir
- On arrive parfois à des logiciels non utilisables

- **En 1983, on se rend compte que :**

- 43% des logiciels sont livrés et non utilisés
- 28% payés et non livrés
- 19% utilisés tels quels
- 3% utilisés sans modification (échec de tentative)
- 2% utilisés avec modifications (réussite de tentative)

- Les coûts de maintenance (lorsque cela se fait) dépassent ceux du développement
- Le coût d'une erreur parfois dépasse le coût du système

Illustration : Les pannes causées par le « **bug de l'an 2000** » ont coûté environ 175 milliards de dollars aux entreprises du monde (*journal « Le Monde »* – 23 oct. 2001).

- **D'une étude sur 487 sites de toutes sortes de développement de logiciels, il ressort que :**

- 70% du coût du logiciel est consacré à sa maintenance
- 42% des modifications sont dues à des demandes de l'utilisateur
- 30% des exigences de spécification changent entre la 1ère édition d'une documentation et la première sortie du produit
- 17% à des changements de format des données (an 2000, 2a 2b ...)
- 12% problèmes d'urgence résolus à la hâte
- 9% débogages de programmes
- 6% à des changements de matériel
- 6% problèmes de documentation
- 4% améliorations de l'efficacité
- 3% autres...

Tous ces différents problèmes (crises du logiciel) vont conduire à la création du Génie Logiciel.

En résumé, le **génie logiciel survient dans les années 1968 suite à la crise du logiciel** qui était caractérisé par plusieurs problèmes ou causes parmi lesquels : **la non fiabilité des logiciels, le non-respect du cahier de charge ; la non maîtrise des coûts, le non-respect des délais de livraison.**

2- OBJECTIF OU DEFIS DU GENIE LOGICIEL

Afin de prévenir les différents problèmes pouvant se ramener à la crise du logiciel, le génie logiciel est préoccupé par plusieurs objectifs par exemple :

- ✓ Le développement des logiciels professionnels et rentables ;
- ✓ La fiabilité et la qualité des logiciels ;
- ✓ La maîtrise du coût de construction et de maintenance ;
- ✓ Le respect des délais de livraison ;
- ✓ Le respect du cahier de charge.

Remarque : L'objectif principal du génie logiciel est de produire des logiciels de haute qualité qui répondent aux besoins des utilisateurs de manière efficace et fiable.

Pour y parvenir, il revient à respecter plusieurs :

- ✓ Techniques (interview, analyse des besoins, planification et suivi...) ;
- ✓ Langages (UML par exemple) ;
- ✓ Méthodes (MERISE par exemple)

3- DEFINITION DU GENIE LOGICIEL

Selon les objectifs du génie logiciel, plusieurs définitions peuvent être arrêtées parmi lesquelles :

- ✧ **Définition 1:** « Le génie logiciel est une discipline d'ingénierie qui s'occupe de tous les aspects de la production de logiciels ». Le génie logiciel est intéressé par les théories, les méthodes et les outils de développement de logiciels professionnels.
- ✧ **Définition 2:** selon l'arrêté du 30 décembre 1983: « ensemble des activités de conception et de mise en œuvre des produits et des procédures tendant à rationaliser la production du logiciel et de son suivi »
- ✧ **Définition 3:** « Le génie logiciel est un domaine des sciences de l'ingénieur dont l'objet d'étude est la conception, la fabrication, et la maintenance des systèmes informatiques complexes».

En bref, le Génie Logiciel est l'ensemble des méthodes, outils et techniques utilisées pour développer et maintenir du logiciel dans le but d'assurer la qualité et la productivité. C'est donc l'art de la construction du logiciel.

La « crise du logiciel » n'est pas complètement résolue aujourd'hui.

Le génie logiciel reste un « **art** » qui demande de la part de l'informaticien une bonne formation aux différentes techniques (le **savoir**), mais également un certain entraînement et de l'expérience (le **savoir-faire**).

4- QUELQUES THEMES OU ASPECTS CLES DU GENIE LOGICIEL

- ✓ **Analyse des besoins** : Comprendre les besoins des utilisateurs et les spécifications du système est une étape cruciale. Cela implique souvent des interactions avec les clients pour définir clairement ce que le logiciel doit accomplir.
- ✓ **Conception du logiciel** : La conception consiste à planifier la structure du logiciel en déterminant l'architecture, les modules, les interfaces, et à prendre des décisions sur les algorithmes et les structures de données.
- ✓ **Développement** : La phase de développement consiste à coder le logiciel en utilisant les langages de programmation appropriés. Les développeurs suivent généralement les spécifications établies lors de la conception.
- ✓ **Tests** : Les tests sont essentiels pour s'assurer que le logiciel fonctionne correctement et répond aux spécifications. Cela inclut des tests unitaires, des tests d'intégration et des tests systèmes.
- ✓ **Maintenance** : Une fois le logiciel déployé, il nécessite souvent des mises à jour, des correctifs de bugs et des améliorations. La maintenance du logiciel vise à assurer sa stabilité et à répondre aux besoins changeants des utilisateurs.
- ✓ **Gestion de projet** : Le génie logiciel comprend également la gestion de projet, qui implique la planification, l'organisation des ressources, la gestion des risques et le suivi du progrès pour s'assurer que le projet est livré à temps et dans les limites du budget.
- ✓ **Qualité du logiciel** : La qualité du logiciel est un aspect crucial du génie logiciel. Cela inclut la fiabilité, la maintenabilité, la performance et la convivialité.
- ✓ **Méthodologies de développement** : Il existe différentes méthodologies de développement de logiciels, telles que le modèle en cascade, le développement agile, et d'autres, qui fournissent des cadres de travail pour la gestion de projets et le **développement de logiciels**.

5- DEFINITION DU MOT LOGICIEL

« *Le logiciel est l'ensemble des programmes, procédés et règles, et éventuellement de la documentation, relatifs au fonctionnement d'un ensemble de traitements de l'information* » (arrêté du 22 déc. 1981). En d'autres termes, **un logiciel** pourra donc être considéré comme un ensemble de programmes informatiques (codes sources, éditables, exécutables) mais également les données qu'ils utilisent et les différents documents se rapportant à ces programmes et nécessaires à leur installation, utilisation, développement et maintenance : spécifications, schémas conceptuels, jeux de tests, mode d'emploi, etc.

Un logiciel ou une application est un ensemble de programmes, qui permet à un ordinateur ou à un système informatique d'assurer une tâche ou une fonction en particulier (exemple : logiciel de comptabilité, logiciel de gestion des prêts).

En conclusion,

Le génie logiciel est une discipline en constante évolution en raison des progrès rapides dans la technologie et des besoins changeants des utilisateurs. Il joue un rôle crucial dans la création de solutions logicielles robustes et fiables pour une variété d'applications, allant des applications mobiles aux systèmes d'exploitation en passant par les applications d'entreprise.

PARTIE 2 : NOTION DE CYCLE DE VIE

1- DEFINITION

Le « **cycle de vie d'un logiciel** » (en anglais Software Life cycle), désigne toutes les étapes du développement d'un logiciel, de sa conception à sa disparition.

2- OBJECTIF, ORIGINE ET ROLE DU CYCLE DE VIE

L'objectif est de permettre de définir des jalons intermédiaires permettant la validation du développement logiciel, c'est-à-dire la conformité du logiciel avec les besoins exprimés, et la vérification du processus de développement, c'est-à-dire l'adéquation des méthodes mises en œuvre.

L'origine de ce découpage provient du constat que les erreurs ont un coût d'autant plus élevé qu'elles sont détectées tardivement dans le processus de réalisation.

Le cycle de vie permet de détecter les erreurs au plus tôt et ainsi de maîtriser la qualité du logiciel, les délais de sa réalisation et les coûts associés.

3- PHASES OU ÉTAPES DU CYCLE DE VIE (PHASES DE DÉVELOPPEMENT D'UN LOGICIEL ET SA DISPARITION)

Le cycle de vie d'un logiciel se compose généralement de plusieurs phases, qui peuvent varier légèrement selon les méthodologies, mais voici les phases typiques :

1. Analyse des Besoins

- Identification et collecte des exigences des utilisateurs.
- Documentation des besoins fonctionnels et non fonctionnels.

2. Conception

- Élaboration de l'architecture du système.
- Création de modèles (comme UML) pour visualiser les composants et leurs interactions.
- Spécification des interfaces utilisateur et des bases de données.

3. Développement (ou Implémentation)

- Écriture du code source selon les spécifications de conception.
- Utilisation de langages de programmation et d'outils de développement.

4. Tests

- Vérification et validation du logiciel pour s'assurer qu'il répond aux exigences.
- Types de tests : tests unitaires, tests d'intégration, tests système et tests d'acceptation.

5. Déploiement

- Mise en production du logiciel dans l'environnement des utilisateurs.
- Installation et configuration du système.

6. Maintenance

- Correction des bogues et mise à jour du logiciel en fonction des retours des utilisateurs.
- Ajout de nouvelles fonctionnalités et amélioration des performances.

7. Retrait (ou Fin de Vie)

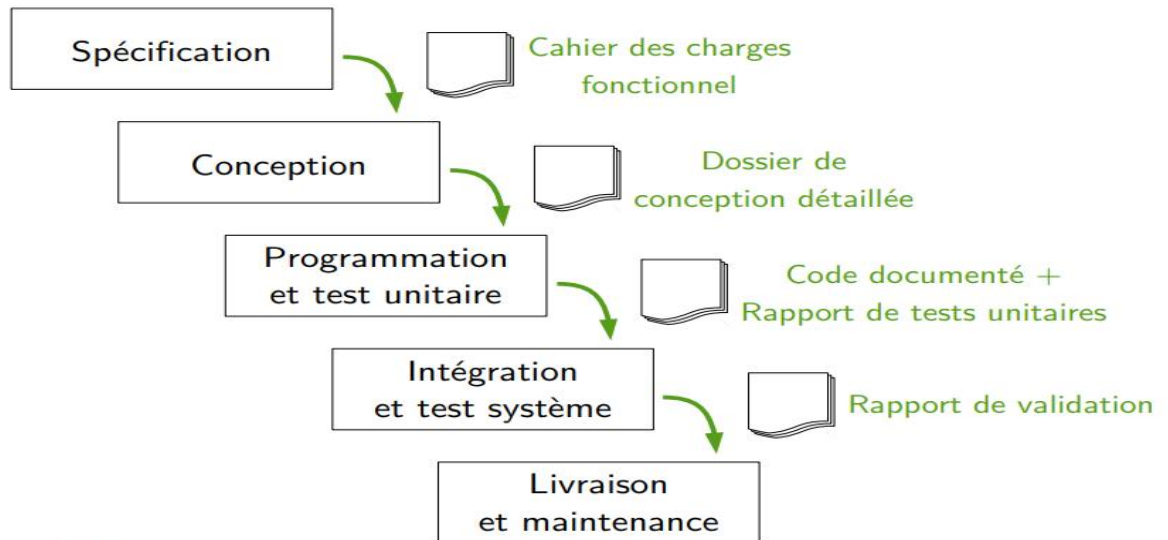
- Planification du retrait du logiciel obsolète.
- Migration des utilisateurs vers un nouveau système, si nécessaire.

NB : Les six premières phases correspondent uniquement aux étapes de développement d'un logiciel qui décrivent la conception détaillée d'un logiciel.

4- QUELQUES MODELES OU METHODES DE CYCLES DE VIE DES LOGICIELS

a) Le modèle en cascade ou waterfall

Chaque étape doit être terminée avant que ne commence la suivante
À chaque étape, production d'un document base de l'étape suivante



Avantages du Cycle de Vie en Cascade

1. Clarté et Structure :

- Chaque phase a des objectifs clairement définis, ce qui facilite la planification et le suivi du projet.

2. Documentation Complète :

- La documentation est produite à chaque étape, ce qui permet une meilleure compréhension et un transfert d'information plus facile entre les membres de l'équipe.

3. Facilité de Gestion :

- La progression linéaire rend la gestion de projet plus simple, avec des jalons clairement identifiables.

4. Convient aux Projets Bien Définis :

- Idéal pour les projets avec des exigences stables et bien comprises, où les changements sont minimes.

Inconvénients du Cycle de Vie en Cascade

1. Rigidité :

- Une fois qu'une phase est terminée, il est difficile de revenir en arrière pour apporter des modifications, ce qui peut poser problème si des exigences changent.

2. Retard dans la Détection des Problèmes :

- Les erreurs ou omissions peuvent ne pas être détectées avant les phases avancées, rendant leur correction plus coûteuse.

3. Manque de Flexibilité :

- Ne s'adapte pas bien aux projets où les exigences évoluent fréquemment, ce qui peut entraîner des frustrations.

4. Tests en Fin de Cycle :

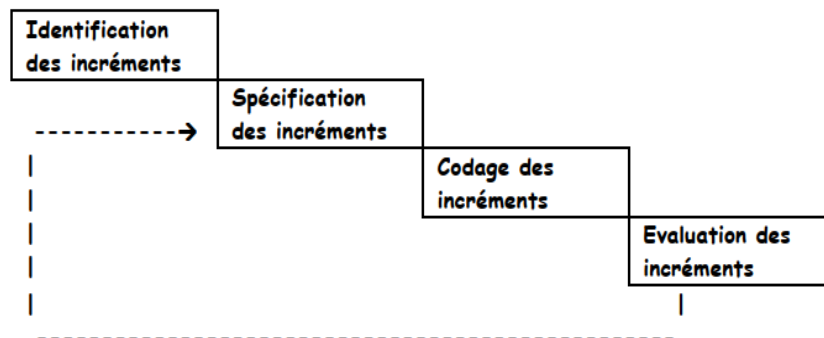
- Les tests sont souvent effectués à la fin du développement, ce qui peut entraîner des découvertes tardives de problèmes critiques.

Conclusion

Le modèle en cascade peut être efficace pour certains types de projets, mais il présente des limitations qui peuvent le rendre moins adapté à des environnements dynamiques ou à des projets nécessitant une grande flexibilité.

b) Le modèle Incrémental

Dans cette méthode, on évolue par incréments, en partant d'un noyau de base qui est par la suite complété progressivement. Généralement, une bonne planification et une spécification rigoureuse sont nécessaires, ainsi qu'une bonne planification et une forte indépendance au niveau fonctionnel et au niveau du calendrier.



Avantages du développement incrémental

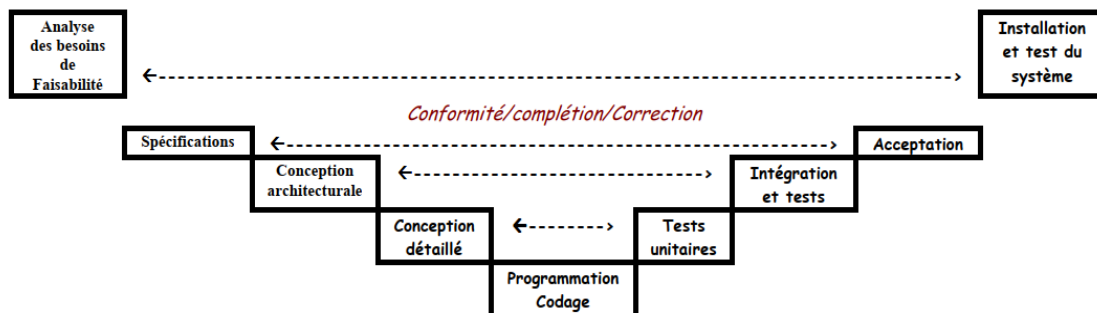
- ✧ Le coût d'accueillir des exigences changeantes des clients est réduit.
 - La quantité de documentation et d'analyse qui doit être refaite est beaucoup moins que ce qui est requis par le modèle en cascade.
- ✧ Il est plus facile d'obtenir les commentaires des clients sur le travail de développement qui a été fait.
 - Les clients peuvent faire des commentaires sur les démonstrations du logiciel et voir ce qui a été mis en œuvre.
- ✧ Plus de livraison rapide et déploiement de logiciels utiles au client est possible.
 - Les clients sont en mesure d'utiliser et gagner de la valeur à partir du logiciel plus tôt que ce qui est possible avec un processus en cascade.

Problèmes de développement incrémental

- ✧ Le processus n'est pas visible.
 - Les gestionnaires ont besoin des produits régulièrement livrables pour mesurer le progrès. Si les systèmes sont développés rapidement, il est pas rentable de produire des documents qui reflètent toutes les versions du système.
- ✧ La structure du système tend à se dégrader que les nouvelles augmentations sont ajoutées.
 - En plus du temps et de l'argent dépensé sur le reconstruction pour améliorer le logiciel, le changement régulier tend à corrompre sa structure. L'intégration de nouveaux changements des logiciels devient de plus en plus difficile et coûteux.

c) Le modèle en V

Le Schéma du modèle en V est présenté ci-dessous.



Dans le modèle en V, chaque spécification faite aux étapes de gauche doit correspondre à une procédure de test (jeu de test à décrire) pour s'assurer que le composant correspond à la description faite aux étapes de droite.

Avantages :

1. Clarté des étapes : Chaque phase a des objectifs clairs et bien définis, facilitant la gestion du projet.
2. Validation précoce : Les tests et la validation sont intégrés au processus, permettant de détecter les erreurs tôt.
3. Documentation complète : La documentation est généralement exhaustive à chaque étape, ce qui facilite la compréhension et la maintenance.
4. Contrôle de qualité : Le modèle met l'accent sur la qualité, avec des tests systématiques à chaque phase.

Inconvénients :

1. Rigidité : Le modèle est peu flexible ; les changements dans les exigences peuvent être difficiles à intégrer une fois le processus lancé.
2. Coûts élevés : Les tests à chaque étape peuvent augmenter les coûts et le temps de développement.
3. Dépendance aux phases précédentes : Si une erreur est détectée tardivement, cela peut nécessiter de revenir à des étapes antérieures, ce qui peut être coûteux en temps et en ressources.

4. Pas adapté aux projets complexes : Pour les projets très dynamiques ou complexes, le modèle en V peut ne pas être le plus efficace.

d) Les méthodes Agiles

Les méthodes Agiles sont un ensemble de pratiques de gestion de projet qui favorisent la flexibilité, la collaboration et l'adaptabilité. Elles se concentrent sur des livraisons itératives et incrémentales, permettant aux équipes de s'ajuster rapidement aux changements de besoins. Les principes fondamentaux incluent la communication constante avec les parties prenantes, l'auto-organisation des équipes et l'accent mis sur la satisfaction du client.

Avantages des Méthodes Agiles

1. Flexibilité et Adaptabilité : Les équipes peuvent réagir rapidement aux changements de priorités et d'exigences, ce qui est essentiel dans un environnement dynamique.
2. Livraisons Rapides : Les fonctionnalités sont livrées par petites itérations, permettant aux clients d'obtenir des résultats rapidement et de fournir des retours en temps réel.
3. Collaboration Renforcée : La communication fréquente entre les membres de l'équipe et les parties prenantes favorise un meilleur alignement et une compréhension commune des objectifs.
4. Satisfaction Client Améliorée : En impliquant les clients tout au long du processus, les méthodes Agiles garantissent que le produit final répond mieux à leurs attentes.

Inconvénients des Méthodes Agiles

1. Manque de Documentation : L'accent mis sur l'interaction peut entraîner une documentation insuffisante, ce qui peut poser des problèmes lors de la maintenance ou du transfert de connaissances.
2. Difficulté de Planification : Les projets peuvent être difficiles à planifier à long terme, car les exigences peuvent évoluer constamment.
3. Ressources Nécessaires : Les méthodes Agiles nécessitent souvent des équipes bien formées et expérimentées, ce qui peut ne pas être toujours disponible.
4. Risque de Sur-Engagement : La pression pour livrer rapidement peut conduire à des surcharges de travail et à des problèmes de qualité si les équipes ne gèrent pas correctement leur charge de travail.

Nous distinguons plusieurs méthodes Agile parmi lesquelles :

1. **Scrum** : Une méthode qui utilise des itérations appelées "sprints" pour livrer des fonctionnalités. Elle se concentre sur des rôles spécifiques (Scrum Master, Product Owner, équipe de développement) et des cérémonies (réunions quotidiennes, revues de sprint, rétrospectives).
2. **Kanban** : Une méthode qui visualise le flux de travail à l'aide d'un tableau Kanban. Elle permet une gestion continue des tâches et se concentre sur l'amélioration continue et la réduction des temps d'attente.

3. **Extreme Programming (XP)** : Une méthode qui met l'accent sur la qualité du code et la satisfaction du client. Elle inclut des pratiques telles que le développement piloté par les tests (TDD), la programmation en binôme et des livraisons fréquentes.
4. **Lean Software Development** : Inspirée des principes du Lean manufacturing, cette méthode vise à maximiser la valeur tout en minimisant le gaspillage. Elle se concentre sur l'efficacité et l'amélioration continue.

Devoir : présenter trois autres modèles de votre choix (définition, schéma, avantages et inconvénients).

PARTIE 3 : QUALITÉ, SPECIFICATION, ERGONOMIE ET TEST

1- QUALITE D'UN LOGICIEL

Pour une meilleure rentabilité et productivité d'un logiciel, il doit répondre aux critères suivants :

- ✓ **Validité** : réponse aux besoins des utilisateurs
- ✓ **Facilité d'utilisation** : prise en main et robustesse
- ✓ **Performance** : temps de réponse, débit, fluidité...
- ✓ **Fiabilité** : tolérance aux pannes
- ✓ **Sécurité** : intégrité des données et protection des accès
- ✓ **Maintenabilité** : facilité à corriger ou transformer le logiciel
- ✓ **Portabilité** : changement d'environnement matériel ou logiciel

2- SPECIFICATION

La spécification est la phase la plus délicate dans le processus de développement du logiciel qui consiste à déterminer exactement ce qui doit être fait.

Nous pouvons distinguer :

- ✓ **Les Spécifications Fonctionnelles**
 - **Description** : Détaille les fonctionnalités et les comportements que le système doit avoir. Elles répondent aux questions de "quoi" le système doit faire.
 - **Exemples** : Cas d'utilisation, scénarios d'utilisateur, exigences de performance.
- ✓ **Spécifications Non Fonctionnelles**
 - **Description** : Énoncent les critères qui jugent le fonctionnement du système, tels que la performance, la sécurité, la fiabilité, et l'utilisabilité. Elles répondent aux questions de "comment" le système doit fonctionner.
 - **Exemples** : Temps de réponse, disponibilité, exigences de sécurité.

Bertrand Meyer [Meyer 90] a défini les 7 pièges qui nous guettent lors d'un processus de spécification, mais également les 7 mots clés de réussite.

a) Les 7 péchés :

- **Bruit** : élément qui n'apporte aucune information (!) / Redondance (!)

- **Silence** : caractéristique d'un problème auquel ne correspond aucune partie du texte
- **Surspécification** : lorsqu'on se lance dans un développement qui donne des détails inutiles (par exemple anticiper sur les tables de la BD quand cela n'est pas nécessaire ...)
- **Contradiction** : dans les séquences de spécification
- **Ambiguïtés** : une phrase peut avoir 2 ou plusieurs sens
- **Référence en avant** : étant à une page, vous faites allusion à un concept qui sera présenté plus loin
- **Vœu pieux** : attention à des éléments non vérifiables (!)

b) Les 7 caractéristiques d'une bonne spécification :

- Conforme (conformité aux besoins réels et non aux besoins supposés exprimés, souvent limités)
- Communicable
- Faisable
- Modifiable
- Validable
- Complète
- Cohérente

3- Les méthodes

Lors de la phase de spécification, nous avons besoin au départ d'interviews bien faites, pouvant déboucher sur une bonne documentation.

Les méthodes de spécification de logiciels seront de trois catégories :

- Informelle (structurée)
- Semi-formelle
- Formelle (mathématique)

Les méthodes de spécification intègrent :

- Modèle
- Démarche
- Langage
- Outils

Remarque :

✓ **Méthodes Informelles**

Elles n'ont pas un formalisme supportant une vérification, une validation (informatique ou simulation mathématique)

Cas de SADT (dans son approche fonctionnelle), BOOCH (dans son approche objet initiale), MERISE...

✓ **Méthodes Semi-formelles**

Elles permettent des vérifications, mais pas sur tous les aspects.

Cas de SADT avec SART, Réseaux de Petri, E/A avec langages ... (entité association)

✓ **Méthodes Formelles (spécification formelle)**

Elles intègrent des langages permettant de définir toutes les contraintes et de garantir la vérification et la validation complète. Ces langages basés sur les mathématiques vont pouvoir prendre en compte n'importe quel concept abstrait ou logique. Les outils sont beaucoup plus orientés vers :

- Automates
- Logique (temporelle, modale)

- Algèbre (algèbre universelle, algèbre de processus)

- Mixte

À titre d'exemple, nous allons citer :

* **VDM (Vienna Description Method)** :

Ce langage utilise la théorie des ensembles pour décrire les entités et les objets ; mais aussi les formules de la

logique de prédicats pour spécifier les propriétés (classes, etc.).

* **Langage Z** :

Z est un langage formel développé à l'Université d'Oxford à la suite des travaux de **Jean René Abrial**. Le langage Z utilise les notions ensemblistes, le calcul des propositions (et, ou, non, implication, etc.) et des prédicats (quantificateurs existentiels – il existe - et universel – quel que soit -), les relations (partie du produit cartésien de plusieurs ensembles) et fonctions (relations avec au plus une image par valeur du domaine de définition), les séquences ou suites (fonctions des entiers naturels dans un autre ensemble pour imposer un ordre aux valeurs).

* **Langage B** :

B est une méthode (incluant le langage B) également développée par J.R. Abrial, qui pourrait substituer **Z**. **B** couvre le processus de la spécification formelle au programme. La méthode **B** dispose d'un langage de spécification à base de machines abstraites, d'une technique de raffinement des spécifications (des notions abstraites aux notions des langages de programmation), des obligations de preuve associées à chaque étape, d'un outil permettant de supporter ce processus (atelier **B**, Steria).

* Méthode **RAISE** :

RAISE (Rigorous Approach for Industrial Software Engineering) est une méthode développement formelle assez rigoureuse, qui possède son propre langage formel spécification [Raise95]

NB :

- ✓ **La vérification** est le processus d'évaluation des produits intermédiaires (comme les spécifications, le code, et la documentation) pour s'assurer qu'ils respectent les exigences spécifiées.
- ✓ **La validation** est le processus d'évaluation du produit final pour s'assurer qu'il répond aux besoins et aux attentes des utilisateurs.
- ✓ **La validation** est effectuée après les vérifications, à la fin du cycle de développement.
- ✓ **L'acceptation** est le processus par lequel les utilisateurs finaux ou les parties prenantes évaluent le produit pour décider s'ils l'acceptent pour une utilisation dans un environnement de production.

4- LES TESTS

✧ Le test est une activité importante dont le but est d'arriver à un produit « zéro défaut ».

✧ C'est la limite idéaliste vers laquelle on tend pour la qualité du logiciel.

✧ Généralement 40% du budget global est consacrée à l'effort de test.

✧ Le test est une recherche d'anomalie dans le comportement de logiciel. C'est une activité paradoxale: il vaut mieux que ce ne soit pas la même personne qui développe et qui teste le soft. D'où le fait qu'un bon test est celui qui met à jour une erreur (non encore rencontrée).

Qu'est-ce qu'un test réussi ?

- ✧ « Un test réussi n'est pas un test qui n'a pas trouvé de défauts, mais un test qui a effectivement trouvé un défaut (ou une anomalie) » G.J. Myers

Erreur, défaut, anomalie

- ✧ Une anomalie (ou défaillance) est un comportement observé différent du comportement attendu ou spécifié. Exemple. Le 4 juin 1996, on a constaté...
 - On constate une anomalie
 - Due à un défaut du logiciel
 - Le défaut est causé généralement par une faute ou erreur (ex. erreur de programmation : confusion d'un « I » avec un « l »)
 - Chaîne de causalité : erreur => défaut => anomalie
 - Nature de l'erreur: spécification, conception, programmation

Tests de programme

- ✧ Les tests visent à montrer qu'un programme fait ce qu'il est censé faire et à découvrir les défauts du programme avant qu'il ne soit mis en service.
- ✧ Lorsque vous testez un logiciel, vous exécutez un programme en utilisant des données artificielles.
- ✧ Vous vérifiez les résultats de l'analyse pour déceler des erreurs, des anomalies ou des informations sur les attributs non fonctionnels du programme.
- ✧ Peut révéler la présence d'erreurs PAS leur Absence.
- ✧ Les tests font partie d'un processus de vérification et de validation plus général, qui comprend également des techniques de validation statique.

Les chiffres montrent que dans un processus de développement, la phase de test coûte 40% de l'ensemble (avec 40% pour l'analyse et 20% pour le codage). Les programmes de tests peuvent parfois être plus longs que le programme à tester. Le concepteur devrait donc y penser dès le départ. Les tests doivent permettre de détecter des erreurs ou des insuffisances dans un logiciel, en se basant sur un référentiel défini au préalable. On retrouvera généralement au niveau opérationnel.

Voici une présentation des principaux types de tests effectués dans le développement logiciel :

1. Tests Unitaires

- **Description** : Vérifient le bon fonctionnement des plus petites unités de code (comme les fonctions ou les méthodes).
- **Objectif** : S'assurer que chaque unité fonctionne comme prévu.
- **Responsabilité** : Généralement effectués par les développeurs.

2. Tests d'Intégration

- **Description** : Évaluent les interactions entre différentes unités ou modules du logiciel.
- **Objectif** : Détecter des problèmes liés à l'intégration des composants.
- **Responsabilité** : Peut être effectué par les développeurs ou une équipe de test.

3. Tests Fonctionnels

- **Description** : Vérifient que le logiciel fonctionne conformément aux spécifications fonctionnelles.
- **Objectif** : S'assurer que toutes les fonctionnalités sont mises en œuvre correctement.
- **Responsabilité** : Généralement effectués par les testeurs.

4. Tests de Système

- **Description** : Évaluent le système dans son ensemble, vérifiant que toutes les fonctionnalités fonctionnent ensemble.
- **Objectif** : S'assurer que le système répond aux exigences spécifiées.
- **Responsabilité** : Équipe de test.

5. Tests d'Acceptation

- **Description** : Réalisés pour déterminer si le logiciel est prêt à être déployé et utilisé par les utilisateurs finaux.
- **Objectif** : Obtenir l'accord des utilisateurs sur la qualité et la conformité du produit.
- **Responsabilité** : Utilisateurs finaux ou parties prenantes.

6. Tests de Performance

- **Description** : Évaluent la réactivité, la vitesse, la scalabilité et la stabilité du système sous une charge donnée.
- **Objectif** : S'assurer que le logiciel peut gérer les exigences de performance dans des conditions réelles.
- **Responsabilité** : Équipe de test spécialisée.

7. Tests de Sécurité

- **Description** : Identifient les vulnérabilités et les failles de sécurité dans le logiciel.
- **Objectif** : Protéger le système contre les menaces et les attaques.
- **Responsabilité** : Équipe de test de sécurité ou experts en sécurité.

8. Tests de Régression

- **Description** : Vérifient que les modifications apportées au code n'ont pas introduit de nouveaux défauts.
- **Objectif** : S'assurer que les fonctionnalités existantes continuent de fonctionner après des changements.
- **Responsabilité** : Équipe de test.

9. Tests de Compatibilité

- **Description** : Évaluent le comportement du logiciel sur différentes plateformes, navigateurs et appareils.
- **Objectif** : S'assurer que le logiciel fonctionne correctement dans divers environnements.
- **Responsabilité** : Équipe de test.

10. Tests Exploratoires

- **Description** : Tests non scriptés où les testeurs explorent le logiciel pour identifier des problèmes.
- **Objectif** : Découvrir des défauts qui pourraient ne pas être couverts par des tests formels.
- **Responsabilité** : Testeurs expérimentés.

11. Tests de Boîte Blanche

Les tests de boîte blanche consistent à tester le logiciel en ayant accès à son code source. Le testeur examine la logique interne et la structure du programme, ce qui lui permet de concevoir des cas de test basés sur la façon dont le code est écrit.

En conclusion, un gros projet doit absolument avoir un **responsable des tests**. Les tests effectués pourront être **statiques** (porter sur des textes, des spécifications ou des modules programmes sans les exécuter) ou **dynamiques** (porter sur des programmes à exécuter ou sur des séries de données entrées).

Quelques conséquences liées aux mauvais tests des applications ;

1. **Défaillances en production** : Des tests insuffisants ou mal réalisés peuvent conduire à des bugs non détectés qui se manifestent en production, entraînant des pannes, des pertes de données ou des interruptions de service.
2. **Perte de confiance des utilisateurs** : Si une application présente des erreurs fréquentes, cela peut nuire à la réputation de l'entreprise et entraîner une perte de confiance de la part des utilisateurs, ce qui peut réduire la fidélité à la marque.
3. **Coûts supplémentaires** : Corriger des erreurs après le déploiement est généralement plus coûteux que de les détecter et de les corriger en phase de test. Cela peut entraîner des dépassements de budget et des retards dans le développement de nouvelles fonctionnalités.

PARTIE 4 : RÉDACTION DE CAHIER DES CHARGES

1- DEFINITION ET ROLE

Le cahier des charges est un document essentiel à l'élaboration et la réalisation d'un projet. Il permet de formaliser les attentes et les besoins du donneur d'ordre (ou de la maîtrise d'ouvrage ou MOA) de manière exhaustive que doit réaliser le MOE.

2- QUELQUES INTERNANTS OU ACTEURS D'UN PROJET

- ✓ **Maître ou maîtrise d'ouvrage (MOA)** : Le MOA est une personne morale (entreprise, direction etc.), une entité de l'organisation.
- ✓ **Maître ou maîtrise d'œuvre (MOE)** : Le MOE est une personne morale (entreprise, direction etc.) garante de la bonne réalisation technique des solutions. Il a, lors de la conception du SI, un devoir de conseil vis-à-vis du MOA, car le SI doit tirer le meilleur parti des possibilités techniques.

Le MOA est client du MOE à qui il passe commande d'un produit nécessaire à son activité.

Le MOE fournit ce produit ; soit il le réalise lui-même, soit il passe commande à un ou plusieurs fournisseurs (« entreprises ») qui élaborent le produit sous sa direction.

La relation MOA et MOE est définie par un contrat qui précise leurs engagements mutuels.

Lorsque le produit est compliqué, il peut être nécessaire de faire appel à plusieurs fournisseurs. Le MOE assure leur coordination ; il veille à la cohérence des fournitures et à leur compatibilité. Il coordonne l'action des fournisseurs en contrôlant la qualité technique, en assurant le respect des délais fixés par le MOA et en minimisant les risques.

Le MOE est responsable de la qualité technique de la solution. Il doit, avant toute livraison au MOA, procéder aux vérifications nécessaires (« recette usine »).

3- REDACTION DU CAHIER DE CHARGE

Les principales rubriques d'un cahier de charge fonctionnel sont :

- Contexte du projet.
- Spécifications non fonctionnelles.
- Spécifications fonctionnelles.
- Ressources.
- Délais.
- Besoins financiers et budget.